

Predictive Maintenance Challenge

Vibe Coding Hackathon - March 4, 2026
UW-Milwaukee Connected Systems Institute

Your Mission

Build a predictive maintenance assistant that warns a maintenance team before a robot breaks down.

The robot: A green collaborative robot arm in CSI's lab that picks and places small parts all day.

Your goal: Give maintenance workers an early warning when something is about to go wrong, and tell them what to do about it.

What You Get

Two data files:

File	What it is
training_data.csv	10,000 rows of normal robot operation (your baseline).
actual_data.csv	10,000 rows with hidden problems (can you find them?).

Each file contains about 2.5 hours of robot operation data, sampled every 1-2 seconds.

Understanding the Data

Think of it like vital signs for a robot. Just like doctors monitor heart rate and temperature, we monitor:

Joint Currents (6 sensors: Current_J0 to Current_J5)

- How much electrical power each joint is using.
- Normal: -6 to +6 Amperes.
- Problems show up as: sudden spikes, gradual increases, or weird patterns.

Joint Temperatures (6 sensors: Temperature_T0, Temperature_J1 to Temperature_J5)

- How hot each joint is getting.
- Normal: 28-45 °C.
- Problems show up as: rapid heating, staying too hot, strange fluctuations.

Joint Speeds (6 sensors: Speed_J0 to Speed_J5)

- How fast each joint is moving.
- Normal: usually near zero (robot moves in short bursts).
- Problems show up as: moving when it should not, not moving when it should.

Tool Current (1 sensor: Tool_current)

- How much power the gripper/end-effector uses.

- Normal: 0.07-0.15 Amperes.
- Problems show up as: inconsistent grip strength, erratic behavior.

Fault Flags (2 sensors: Robot_ProtectiveStop, grip_lost)

- Safety stops: robot detected something wrong and stopped itself.
- Grip failures: robot dropped the part it was holding.
- Normal: very rare (maybe 10-20 times in 10,000 operations).
- Problems show up as: happening repeatedly in a short time.

Other Info

- Num: row number (1 to 10,000).
- Timestamp: when this data was recorded.
- cycle: operation cycle count (tracks total robot usage).

What Problems Are Hidden in the Data?

The anomalies file contains 9 different types of problems that real robots face:

Easy to Spot

- Electrical overload - a motor suddenly pulls way too much power.
- Overheating - temperature shoots up really fast.
- Repeated safety stops - robot keeps stopping itself over and over.

Medium Difficulty

- Wear and tear - power usage slowly creeps up over time.
- Sensor failure - robot reports it is moving but using no power (impossible).
- Electrical noise - random spikes and jitter in the readings.

Hard to Spot

- Control issues - tiny repeating patterns in temperature.
- Coordination problems - two joints that should work together start fighting each other.
- Early degradation - very subtle changes that hint at future problems.

Your job: Build something that can detect these problems before they cause a breakdown.

What You Need to Build

To complete the challenge, your demo must have:

1) Use real data

- Load and analyze the CSV files.
- Do not just show a mockup - actually process the data.

2) Give early warning

- "High risk of motor failure detected."
- "Recommend maintenance in the next 50 cycles."

- "Joint 3 temperature trending toward critical - check cooling system."
- Focus on prevention, not just reporting what already happened.

3) Include AI interaction

- Chat: "What's wrong with Joint 3?"
- Summary: "Detected 3 anomalies today - 2 thermal, 1 electrical."
- Q&A: "Should we schedule maintenance?"

4) Make it easy to use

- Dashboard, web page, or visual display.
- Maintenance workers should understand it without a PhD.
- Clear, actionable information.

Ideas to Get Started

Approach 1: Threshold Alerts

Compare current data to a healthy baseline:

- "If temperature > 50 °C -> ALERT"
- "If current > 5 A -> WARNING"
- "If 3+ safety stops in 2 minutes -> PROBLEM"

Approach 2: Trend Detection

- "Temperature rising 5 °C per minute"
- "Current usage up 20% from baseline"
- "Grip failures increasing"

Approach 3: Smart Assistant

- "I see Joint 3 is overheating. This usually means the cooling fan needs checking."
- "Multiple protective stops suggest an obstacle in the robot's path."

Approach 4: Health Score

- "System Health: 87/100 - Good"
- "Predicted time to maintenance: 3 days"
- "Risk level: LOW"

Quick Start (for coders)

```
import pandas as pd

# Load the data
healthy = pd.read_csv('dataset_healthy_10k.csv')
test = pd.read_csv('dataset_with_anomalies.csv')

# Learn what's normal
normal_temp = healthy['Temperature_J1'].mean()
normal_current = healthy['Current_J1'].mean()

# Check for problems
if test['Temperature_J1'].max() > normal_temp + 10:
    print("Overheating detected!")

if test['Current_J1'].max() > normal_current + 3:
    print("Current spike detected!")
```

What Judges Look For

- Does it work? Can you demo it live?
- Is it useful? Would a real maintenance team use this?
- Does it predict? Or does it just describe what already happened?
- Is it clear? Can anyone understand the output?
- How creative? Did you find an interesting approach?

You do not need to catch all 9 problems. Detecting 2-3 problems well is impressive.

Tools You Can Use

Programming languages: Python, JavaScript, anything you like.

AI tools: ChatGPT, Claude, Copilot, Cursor - all allowed and encouraged.

Libraries (if using Python):

- pandas - for reading CSV files.
- matplotlib / plotly - for charts.
- streamlit / gradio - for quick web interfaces.
- scikit-learn - for machine learning.

Do not know how to code? Partner with someone who does, or use AI to help write code.

Timeline

8:00 AM	Challenge starts, data available.
3:00 PM	Stop coding, prepare demo.
3:00-5:00 PM	Judging rounds.
5:00 PM	Winners announced.

Prizes

- Grand Prize: \$2,000

- Best Technical: \$1,000
- Best Applied: \$1,000
- Finalists (2): \$500 each

All 5 finalists are guaranteed a prize.

Final Tips

Do this:

- Start simple - get something working first.
- Focus on 1-2 types of problems, not all 9.
- Make your demo easy to understand.
- Practice your 3-minute explanation.

Do not do this:

- Try to build a perfect solution.
- Get stuck on one complex algorithm.
- Make something only you can understand.
- Forget to actually use the data files.

The Bottom Line

Build something you can demo in 3 minutes that:

- Uses the robot data.
- Warns about problems before they happen.
- Includes some AI interaction.
- Actually works.

Accuracy does not matter. Finding 3 problems is better than finding 0 problems perfectly.

Good luck!

Questions? Ask mentors on-site or check Canvas.

Remember: vibe coding is encouraged. Use AI. Move fast. Build something cool.

Dataset prepared for UW-Milwaukee CSI Predictive Maintenance Challenge.